

```
NumPy Array: [4 5 6]
Converted PyTorch Tensor: tensor([4, 5, 6])
Type: <class 'torch.Tensor'>
```

PyTorch Autograd Notes

1. Need for Autograd

Explanation

- **Autograd** = Automatic Differentiation in PyTorch. It calculates derivatives automatically.
- Derivatives are crucial for training neural networks (backpropagation).
- **Problem:** As neural networks become deeper (nested functions), manually computing derivatives using chain rule becomes extremely difficult.

Neural Network as Nested Function

A simple neural network (single neuron with sigmoid) involves:

1. `z = w * x + b`
2. `y_pred = sigmoid(z)`
3. `Loss = BinaryCrossEntropy(y_pred, y_true)`

Here, Loss is a nested function of w and b. To update w and b, we need:

- $\partial L / \partial w$ and $\partial L / \partial b$ using chain rule:
 - $\partial L / \partial w = \partial L / \partial y_{\text{pred}} * \partial y_{\text{pred}} / \partial z * \partial z / \partial w$
 - $\partial L / \partial b = \partial L / \partial y_{\text{pred}} * \partial y_{\text{pred}} / \partial z * \partial z / \partial b$

Manually coding these for large networks is impractical. **Autograd automates this.**

2. Autograd Basics

Explanation

- Set `requires_grad=True` on a tensor to track operations on it.
- After forward pass, call `.backward()` on the output tensor.
- Gradients are stored in `.grad` attribute of the input tensor.
- PyTorch builds a **computation graph** (Directed Acyclic Graph) during forward pass.
- Backward pass traverses the graph from root (output) to leaves (inputs) applying chain rule.

Code & Output

```
import torch

x = torch.tensor(3.0, requires_grad=True)    # leaf tensor
y = x ** 2                                   # operation
print("x:", x)
print("y:", y)
```

Output:

```
x: tensor(3., requires_grad=True)
y: tensor(9., grad_fn=<PowBackward0>)

y.backward()                                # compute gradients
print(x.grad)                               # dy/dx = 2*x = 6
```

Output:

```
tensor(6.)
```

3. Example 1: $y = x^2$

Explanation

- We want dy/dx at any x .
- Using Autograd: define x with `requires_grad=True`, compute y , call `y.backward()`, get `x.grad`.

Code & Output

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"dy/dx at x=3: {x.grad}")    # 2*3 = 6

# For x=4
x = torch.tensor(4.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"dy/dx at x=4: {x.grad}")    # 2*4 = 8
```

Output:

```
dy/dx at x=3: 6.0  
dy/dx at x=4: 8.0
```

4. Example 2: Nested Functions ($z = \sin(x^2)$)

Explanation

- Given: $y = x^2$, $z = \sin(y)$. Need dz/dx .
- Chain rule: $dz/dx = dz/dy * dy/dx = \cos(y) * 2x = \cos(x^2) * 2x$.
- Autograd handles it automatically.

Code & Output

```
import torch  
  
x = torch.tensor(3.0, requires_grad=True)  
y = x ** 2  
z = torch.sin(y)  
z.backward()  
print(f"dz/dx at x=3: {x.grad}")    # cos(9)*6 ≈ -5.46? Actually  
cos(9)*6 ≈ -5.46? Let's compute: cos(9 rad) ≈ -0.911, times 6 = -5.466  
  
# For x=4  
x = torch.tensor(4.0, requires_grad=True)  
y = x ** 2  
z = torch.sin(y)  
z.backward()  
print(f"dz/dx at x=4: {x.grad}")    # cos(16)*8 ≈ -7.66
```

Output:

```
dz/dx at x=3: -5.466... (approx)  
dz/dx at x=4: -7.66... (approx)
```

Note: Gradients are only stored for leaf tensors (inputs). Intermediate tensors (like `y`) have `None` for `.grad`.

5. Example 3: Single Neuron Neural Network

Explanation

- Dataset: CGPA (x) $\rightarrow$ Placement (y_{true}). Example: $x=6.7$, $y_{\text{true}}=0$.
- Model: $z = w*x + b$, $y_{\text{pred}} = \text{sigmoid}(z)$, Loss = Binary Cross Entropy.
- Need $\partial L / \partial w$ and $\partial L / \partial b$ to update weights.
- **Manual approach:** Derive and code chain rule explicitly (tedious).

- **Autograd approach:** Define forward pass, call `loss.backward()`, get gradients.

Manual Code (for comparison)

```
# Manual derivatives (conceptual)
x = 6.7
y_true = 0.0
w = 1.0
b = 0.0

# Forward
z = w * x + b
y_pred = 1 / (1 + torch.exp(-z)) # sigmoid
loss = -(y_true * torch.log(y_pred) + (1 - y_true) * torch.log(1 - y_pred))

# Manual gradients (chain rule)
dL_dy_pred = (y_pred - y_true) / (y_pred * (1 - y_pred))
dy_pred_dz = y_pred * (1 - y_pred)
dz_dw = x
dz_db = 1
dL_dw = dL_dy_pred * dy_pred_dz * dz_dw # = (y_pred - y_true) * x
dL_db = dL_dy_pred * dy_pred_dz * dz_db # = (y_pred - y_true)
```

Autograd Code

```
import torch

# Data
x = torch.tensor(6.7)
y_true = torch.tensor(0.0)

# Parameters with gradient tracking
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

# Forward pass
z = w * x + b
y_pred = torch.sigmoid(z)
loss = torch.nn.functional.binary_cross_entropy(y_pred, y_true) # or manual formula

# Backward pass (automatic gradients)
loss.backward()

# Gradients
print(f"∂L/∂w = {w.grad}") # (y_pred - y_true) * x
print(f"∂L/∂b = {b.grad}") # (y_pred - y_true)
```

Output:

```
∂L/∂w = tensor(6.6999)
∂L/∂b = tensor(1.0000)    # approx (y_pred - 0) with y_pred near 0.999?
```

Benefit: No manual chain rule coding.

6. Working with Vector Inputs

Explanation

- When input is a vector, output is a **multivariable function**.
- Gradient w.r.t each input variable is computed separately.
- Example: $y = \text{mean}(x^2)$ where $x = [x_1, x_2, x_3]$.

Code & Output

```
import torch

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = (x ** 2).mean()    # (1^2 + 2^2 + 3^2)/3 = (1+4+9)/3 = 14/3 ≈ 4.6667
y.backward()
print(x.grad)          # dy/dx = [2*x1/3, 2*x2/3, 2*x3/3] = [0.6667, 1.3333, 2.0]
```

Output:

```
tensor([0.6667, 1.3333, 2.0000])
```

7. Clearing Gradients

Explanation

- Calling `.backward()` multiple times **accumulates** gradients (adds new gradients to existing `.grad`).
- Always **zero out** gradients before next backward pass to avoid incorrect values.
- Use `x.grad.zero_()` (in-place operation).

Code & Output (Problem)

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)    # 6.0

# Second forward+backward without clearing
```

```
y = x ** 2
y.backward()
print(x.grad)    # 6.0 + 6.0 = 12.0 (WRONG for second pass)
```

Output:

```
tensor(6.)
tensor(12.)
```

Solution

```
x = torch.tensor(3.0, requires_grad=True)

# First pass
y = x ** 2
y.backward()
print(x.grad)    # 6.0

# Clear gradients
x.grad.zero_()

# Second pass
y = x ** 2
y.backward()
print(x.grad)    # 6.0 (correct)
```

Output:

```
tensor(6.)
tensor(6.)
```

8. Disabling Gradient Tracking

Explanation

- During **inference** (prediction) we don't need gradients – only forward pass.
- Disabling saves memory and computation.
- Three methods:
 - a. `requires_grad_(False)` – in-place change on tensor.
 - b. `detach()` – creates a new tensor detached from graph.
 - c. `with torch.no_grad():` – context manager (most convenient).

Code & Output

Method 1: `requires_grad_(False)`

```
x = torch.tensor(3.0, requires_grad=True)
x.requires_grad_(False)
print(x.requires_grad)  # False
y = x ** 2
# y.backward() would raise error because no gradient tracking
```

Method 2: `detach()`

```
x = torch.tensor(3.0, requires_grad=True)
z = x.detach()  # z has same data but no gradient tracking
print(z.requires_grad)  # False
y = z ** 2
# y.backward() -> RuntimeError
```

Method 3: `with torch.no_grad():` (Recommended)

```
x = torch.tensor(3.0, requires_grad=True)
with torch.no_grad():
    y = x ** 2
print(y.requires_grad)  # False
# y.backward() -> Error
```

Output (for method 3):

False

Summary

Concept	Key Code
Enable gradient	<code>torch.tensor(..., requires_grad=True)</code>
Forward pass	<code>y = f(x)</code>
Backward pass	<code>y.backward()</code>
Get gradient	<code>x.grad</code>
Zero gradients	<code>x.grad.zero_()</code>
Disable gradients (inference)	<code>with torch.no_grad():</code>

Autograd is essential for training neural networks in PyTorch. It automatically builds the computation graph and computes gradients via backpropagation, saving you from manual chain rule calculations.